💅 ClavusOS — Operação

Ficha de bancada: variáveis, comandos, studio novo, Asaas em dois níveis e o checklist de publicação.
Produto da AF Tecnologia e Marketing · repositório **alexfiorotti-af/clavusOS** · gerado em 28/10/2026

> Nomes de variáveis e comandos, nunca valores. Mudou algo aqui? Atualiza no mesmo commit.

1. Como roda

- **Código:** `app/` — Node 22, TypeScript direto (`--experimental-strip-types`), `node:http`, `pg`. Única dependência de produção: o driver `pg`.
- **Banco:** PostgreSQL 14+ (Supabase, região São Paulo). Migrações em `db/0NN_*.sql`; `npm run migrar` aplica as que faltam (ou cole no editor SQL do Supabase, "Run without RLS").
- **Entrada na hospedagem:** `package.json` da raiz (`npm start` → `app/src/servidor.ts`).
- **Todos os `npm run` são de dentro de `app/`.**
- **Protótipo da Fase 1** (PWA offline) vive em `prototipo/` — é demo comercial, não produto.

2. Variáveis de ambiente (painel da hospedagem ou `app/.env` local)

VARIÁVEL	O QUE É
`CLAVUS_DATABASE_URL`	conexão do Postgres (usuário dono do esquema; o app assume `clavus_app` em cada transação)
`CLAVUS_DOMINIO_BASE`	domínio do produto, ex. `clavusos.com.br` — o studio é `<slug>.<domínio>`
`CLAVUS_STUDIO_PADRAO`	slug usado quando o Host não traz subdomínio (só instalação de um studio; em produção, vazia — a raiz é a vitrine)
`CLAVUS_WHATSAPP`, `CLAVUS_EMAIL_COMERCIAL`, `CLAVUS_PRECO_MENSAL`, `CLAVUS_DEMO_SLUG`, `CLAVUS_DEMO_SENHA`	a vitrine da raiz
`CLAVUS_ASAAS_AF_CHAVE`, `CLAVUS_ASAAS_AF_AMBIENTE` (`sandbox` | `producao`), `CLAVUS_ASAAS_AF_WEBHOOK_TOKEN`	a **conta Asaas da AF**, que cobra a mensalidade do ClavusOS (tela Cobrança). Sem a chave, o botão Assinar vira "fale com a AF"
`CLAVUS_TESTE_DIAS` (7), `CLAVUS_CARENCIA_DIAS` (5)	teste gratuito e carência antes do bloqueio
`CLAVUS_CHAVE_MESTRA`	64 hex; cifra a chave Asaas de cada studio (AES-256-GCM). **Gerar uma vez e nunca trocar sem migrar os segredos**

VARIÁVEL	O QUE É
`CLAVUS_CHAVE_ROTINA`	segredo da rota `/rotina/diaria?chave=…` e do `/rotina/exportar`
`CLAVUS_CONFIA_PROXY`	`true` atrás da Cloudflare (lê `CF-Connecting-IP` e `X-Forwarded-Host`)
`CLAVUS_MODO`	`producao` | `desenvolvimento` | `teste`
`RESEND_API_KEY`	e-mail; sem ela, nada sai (modo dev só registra no log)
`CLAVUS_EMAIL_REMETENTE`	`ClavusOS <avisos@clavusos.com.br>` — o nome visível vira "Studio X via ClavusOS"
`PORT`	porta (a hospedagem define)

Gerar chave-mestra/rotina: `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`.

3. Comandos (`cd app` antes)

COMANDO	PARA QUÊ
`npm run migrar`	aplica migrações pendentes
`npm run criar-studio -- --slug x --nome "Studio X" --admin-nome "Ana" --admin-email ana@x.com [--teste-dias 30] [--cortesia]`	cria o studio (7 dias de teste por padrão) e imprime o link de convite da administradora
`npm run studios`	lista studios; `-- --suspender x` / `--ativar x` / `--cortesia x` / `--pago-ate x AAAA-MM-DD` / `--teste-ate x AAAA-MM-DD`
`npm run studio-modelo`	recria o studio fictício `modelo` (Studio Lume; `demo@<domínio>` / `--senha`, padrão `demonstracao`)
`npm run teste`	puros + de banco (`CLAVUS_TESTE_DATABASE_URL`, padrão `postgres://localhost/clavus_teste` — **recria o esquema**)
`npm run tipos`	`tsc --noEmit`
`npm start`	sobe o servidor

4. Endereços: raiz, studios e demonstração

- **Raiz** (`CLAVUS_DOMINIO_BASE` sem subdomínio): a vitrine do produto, sem studio.
- **Studio** = `<slug>.<domínio>`. `CLAVUS_STUDIO_PADRAO` só serve a instalação de um studio; em produção fica **vazia**. Em desenvolvimento, `?studio=slug` também resolve.
- **Demonstração** = `modelo.<domínio>` (Studio Lume, fictício, plano cortesia): entradas `demo@<domínio>` (administradora) e `equipe@<domínio>` (profissional), senha `CLAVUS_DEMO_SENHA`.
- **Página pública da cliente** = `<slug>.<domínio>/agendar` (liga/desliga em Configurações).

4a. Studio novo = três passos

1. `npm run criar-studio -- --slug x --nome "..." --admin-nome "..." --admin-email ...` → convite de 3 dias.
2. **DNS:** o curinga `*` no domínio do produto cobre qualquer slug. Na Hostinger, estacionar `<slug>.<domínio>` como **alias** no site do app (nunca "criar subdomínio" — lição D010 do Pousa).
3. Mandar o link de convite à dona no WhatsApp. Ela cadastra serviços em Configurações.

5. Rotina (o relógio de fora)

Cloudflare Worker (ou cron da hospedagem) chama a cada hora: `GET https://<domínio>/rotina/diaria?chave=<CLAVUS_CHAVE_ROTINA>` Para cada studio: sincroniza cobranças pendentes com o Asaas (studio e AF), marca vencidas, manda o lembrete de amanhã às clientes com e-mail, avisa vencimento da mensalidade do ClavusOS (3 dias antes, no dia, na carência) e limpa sessões. Idempotente: chamar duas vezes não repete nada.

5a. Backup enquanto o Supabase for Free

`GET /rotina/exportar?chave=…` lista os slugs; `&slug=x` devolve o JSON completo do studio (formato `clavusos-exportacao/1`, sem senhas/chaves/tokens). Agendar no PC (tarefa diária) + `pg_dump` semanal. Some quando o Supabase Pro entrar (backup diário próprio). A administradora baixa os dados do próprio studio em Configurações → Exportar (fica na auditoria).

6. Asaas em dois níveis

- **Do studio (a cliente paga o studio):** a administradora cola a chave em Configurações → Asaas (guardada **cifrada**; sandbox ou produção; a chave é testada antes de entrar). Webhook no Asaas: `https://<slug>.<domínio>/webhooks/asaas/<token>` (o token aparece na tela). Eventos: `PAYMENT_RECEIVED, PAYMENT_CONFIRMED, PAYMENT_OVERDUE, PAYMENT_REFUNDED, PAYMENT_DELETED`.
- **Da AF (o studio paga o ClavusOS):** webhook `https://<domínio raiz>/webhooks/asaas-af/<CLAVUS_ASAAS_AF_WEBHOOK_TOKEN>` cadastrado na conta Asaas da AF (o token também é aceito no cabeçalho `asaas-access-token`).
- **Regra da casa:** o webhook **nunca** dá baixa sozinho; o sistema consulta a cobrança na API antes.
- Assinaturas de cliente usam o ciclo nativo do Asaas: mensal `MONTHLY`, trimestral `QUARTERLY`, semestral `SEMIANNUALLY`, anual `YEARLY`.

7. Endereços que não são de studio

`/saude` · `/assets/*` · `/rotina/diaria` · `/rotina/exportar` · `/webhooks/asaas-af/<token>`. Tudo o mais exige o slug no Host (ou o studio padrão).

8. Publicar (o checklist do deploy)

1. **Supabase** (região São Paulo): criar o projeto, pegar a **connection string de sessão** (pooler, porta 5432) → `CLAVUS_DATABASE_URL`. Rodar `npm run migrar` (ou colar `db/001` e `db/002` no editor SQL, "Run without RLS").
2. **Hospedagem Node** (Hostinger com suporte a Node, ou VPS): subir o repositório, apontar a entrada para `npm start` na raiz, preencher as variáveis do §2 (`CLAVUS_MODO=producao`).
3. **Domínio**: apontar o domínio do produto e o curinga `*` para a hospedagem; HTTPS automático.
4. **Resend**: conta + domínio verificado → `RESEND_API_KEY` e `CLAVUS_EMAIL_REMETENTE`.
5. **Asaas da AF**: chave da conta AF (sandbox primeiro) + webhook do §6.
6. **Rotina**: Cloudflare Worker/cron chamando `/rotina/diaria` a cada hora.
7. `npm run studio-modelo` para a demonstração; `npm run criar-studio` para o primeiro studio real.

9. Roteiro de 503/500

1. `/saude` responde? Se não: processo caiu → logs da hospedagem.
2. "Faltam variáveis de ambiente" no log → §2.
3. Banco: `CLAVUS_DATABASE_URL` certa? Supabase pausado (plano Free)?
4. Studio novo abrindo a vitrine → alias/curinga do §4a.